

Assignment 6: GUI-Based Multi-Client Chat Application Using TCP

Roll No: 23010348148

Name: Shubojee Mondal

1. Objective

This assignment converts the terminal-based TCP chat client built in Assignment 5 into a graphical desktop application using Python's Tkinter library, while reusing the Assignment 5 server without modification. The goal is to demonstrate GUI programming, event-driven design, multithreaded network I/O, and a responsive user interface layered on top of an existing socket-based protocol.

2. GUI Design

The client (client_gui.py) presents a login window followed by a main chat window. The login window collects the server IP, a username, and an optional password, and validates that the username is not empty before connecting. The chat window contains a scrollable, color-coded message area, a live online-users list, a message entry box with a Private checkbox for one-to-one messaging, Send and Disconnect buttons, and a status bar with a connection indicator. A File/Help menu bar is also provided (Disconnect, Exit, Commands, About).

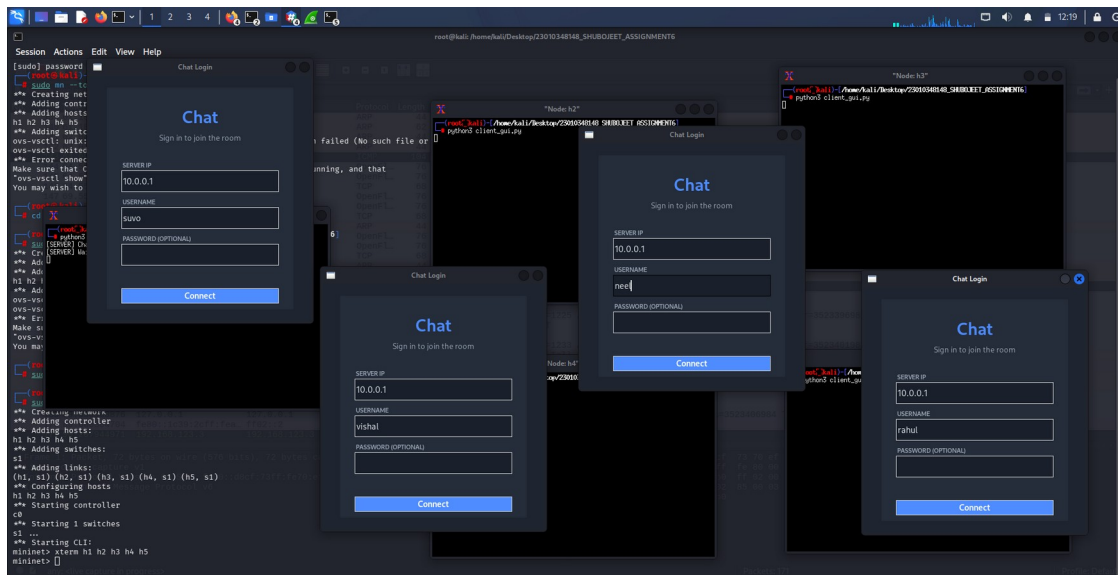


Figure 2.1: Login window for four Mininet clients (suvo, neel, vishal, rahul).

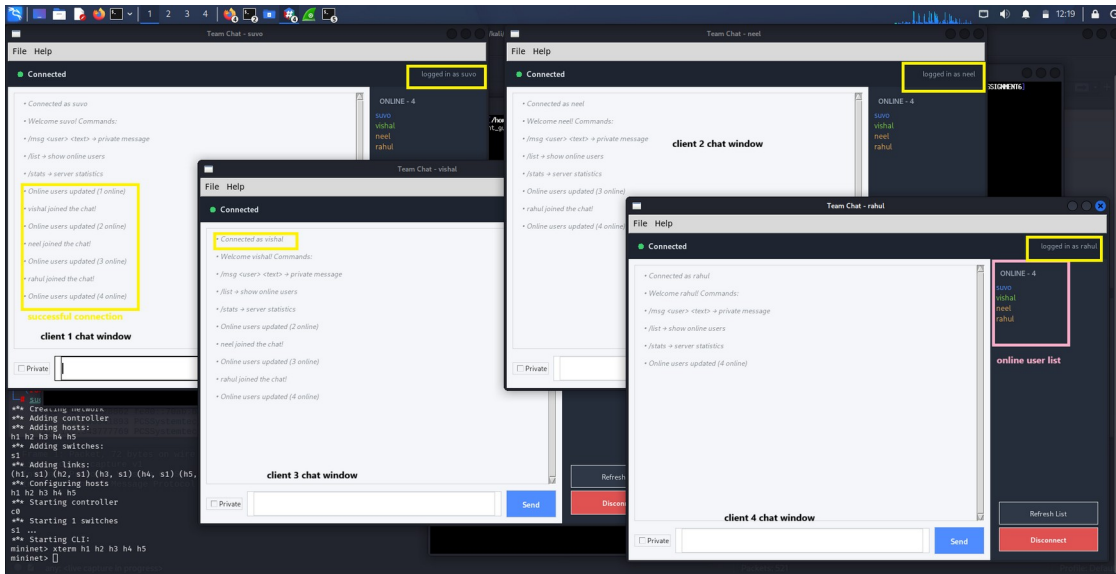


Figure 2.2: Successful connection — server welcome message and progressive join notifications.

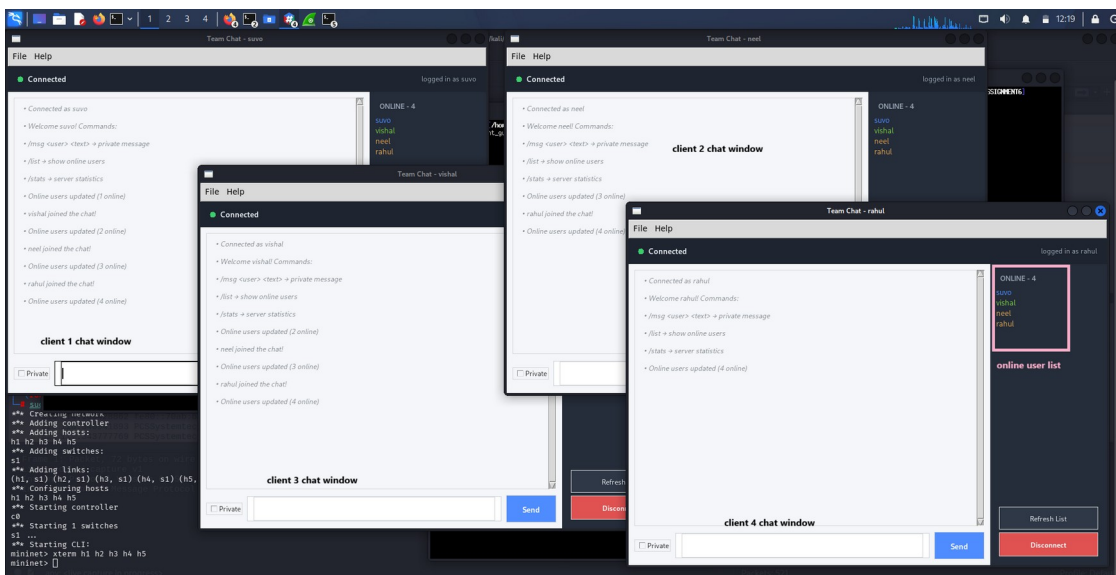
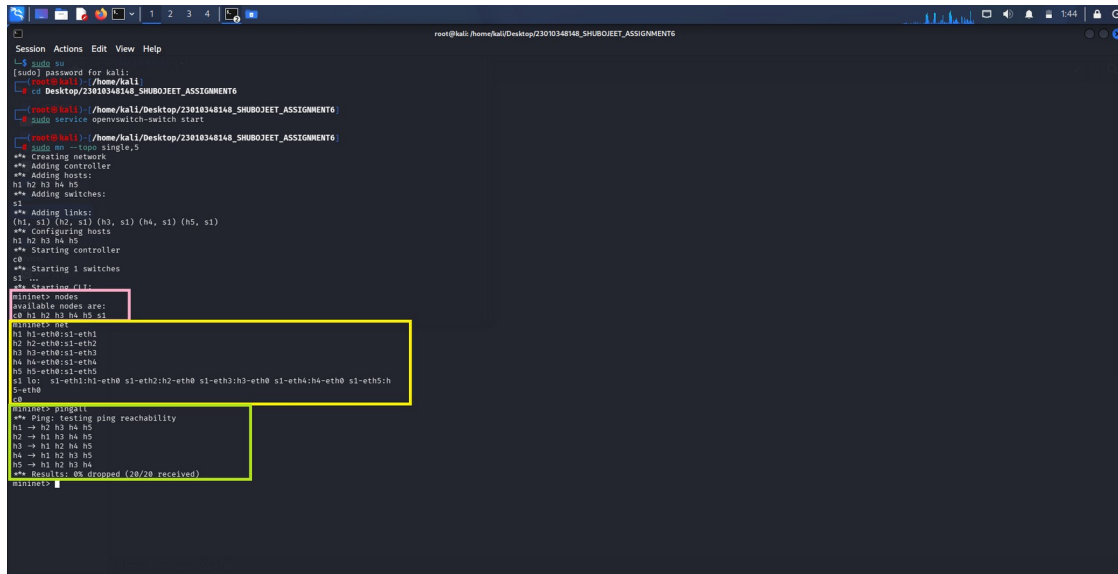


Figure 2.3: Main chat windows for all four connected clients, online-user list populated.

3. System Architecture

The system keeps the Assignment 5 client-server architecture unchanged: a single multi-threaded server (server.py) listens on port 5000 and spawns one thread per connected client, handling broadcast messages, private messages (/msg), the online user list (/list), server statistics (/stats), and persistent chat history (chat_history.csv). Only the client side changes: client_gui.py replaces the terminal input/output loop with a Tkinter interface, while reusing the exact same socket protocol as the Assignment 5 terminal client. The network topology was emulated using Mininet's single,5 topology — one server host (h1) and four client hosts (h2-h5) connected through a single Open vSwitch.



```
Session Actions Edit View Help
root@kali: ~/Desktop/23010348148_SHUBOJIEET_ASSIGNMENT6
~$ sudo su
[sudo] password for kali:
root@kali: ~/Desktop/23010348148_SHUBOJIEET_ASSIGNMENT6
root@kali: ~/Desktop/23010348148_SHUBOJIEET_ASSIGNMENT6
~$ sudo service openvswitch-switch start
sudo service openvswitch-switch start
root@kali: ~/Desktop/23010348148_SHUBOJIEET_ASSIGNMENT6
~$ sudo mininet create
mininet> create
mininet> net
mininet> ipnet
mininet> pingall
mininet>
```

Figure 3.1: Mininet topology verification — nodes, net, and pingall showing 0% packet loss across all five hosts.

4. Components Reused from Assignment 5

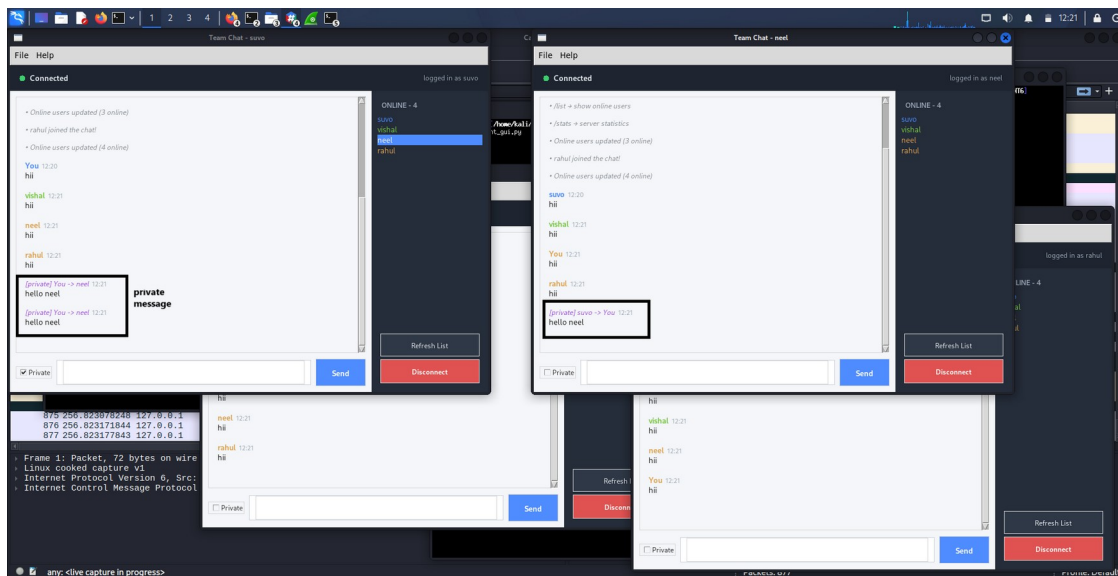
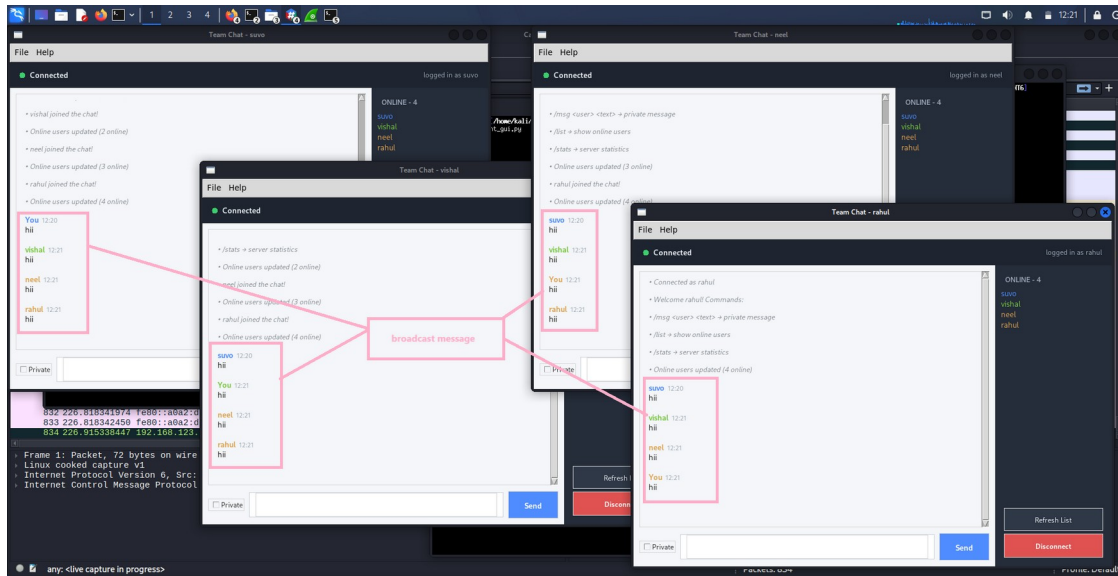
- server.py — used without any modification.
- Socket communication logic — connect, send, and receive calls are identical to the Assignment 5 terminal client.
- Broadcast messaging — server-side broadcast() function and message format are unchanged.
- Private messaging (/msg) — same command syntax and server-side routing via send_private().
- Online user list (/list) and server statistics (/stats) — same server responses, now parsed and rendered graphically.
- Persistent chat history and reconnect replay — chat_history.csv logging and the 'last five messages' replay on reconnect are reused as-is.

5. GUI Design Decisions

- Networking and GUI code are kept in separate classes (ChatClientNetwork vs LoginWindow/ChatWindow) so the socket logic has no dependency on Tkinter.
- A background thread runs the blocking sock.recv() loop; incoming data is placed on a thread-safe queue.Queue rather than touching widgets directly from that thread.
- The main GUI thread polls the queue every 100 ms via root.after(), so it never blocks and the user can keep typing while messages arrive.
- Usernames are colour-coded (hashed to a fixed palette) for readability in the shared chat log.
- Manually typed slash-commands (/list, /msg, /stats) are routed as raw text to the server rather than being echoed as ordinary chat bubbles, matching server-side command parsing exactly.

6. Testing Results

The application was tested end-to-end inside Mininet with four simultaneous clients (suvo, neel, vishal, rahul) connected to the server on h1. All required scenarios were verified in a single continuous session between 12:19 and 12:22, cross-checked against the server's chat_history.csv:



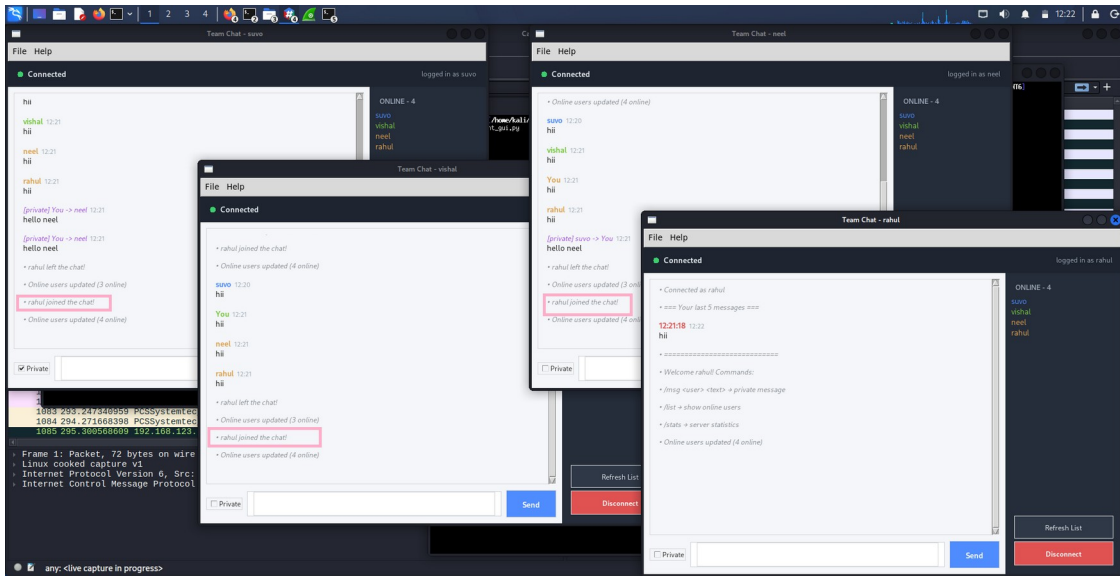


Figure 6.3: User join notification — rahul reconnects and the server automatically replays his last message ("hii" at 12:21:18) via the reconnect-history feature.

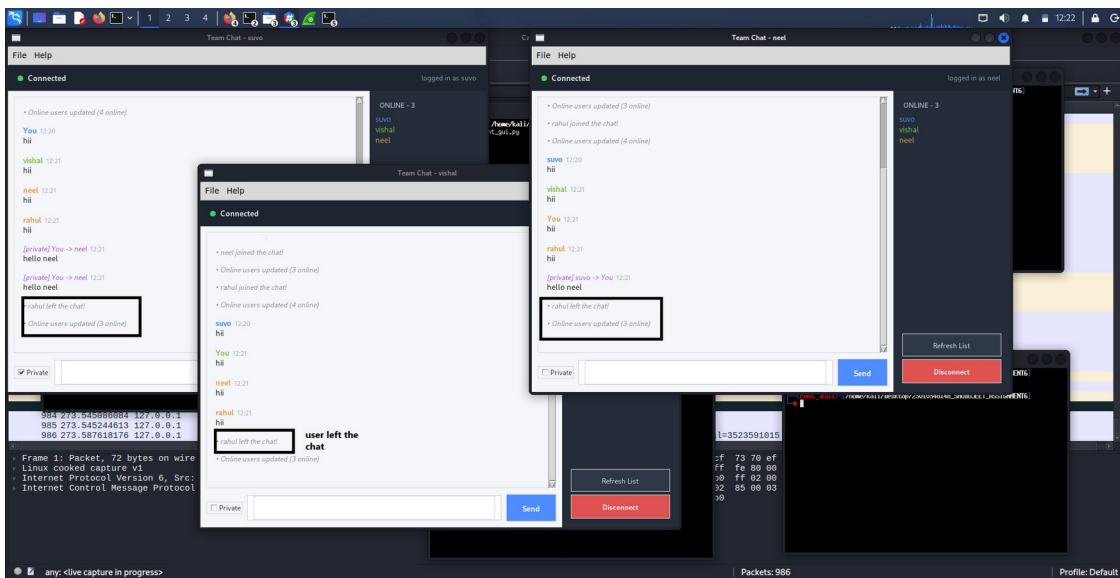


Figure 6.4: User leave notification — "rahul left the chat!" broadcast to remaining clients, online count updates from 4 to 3.

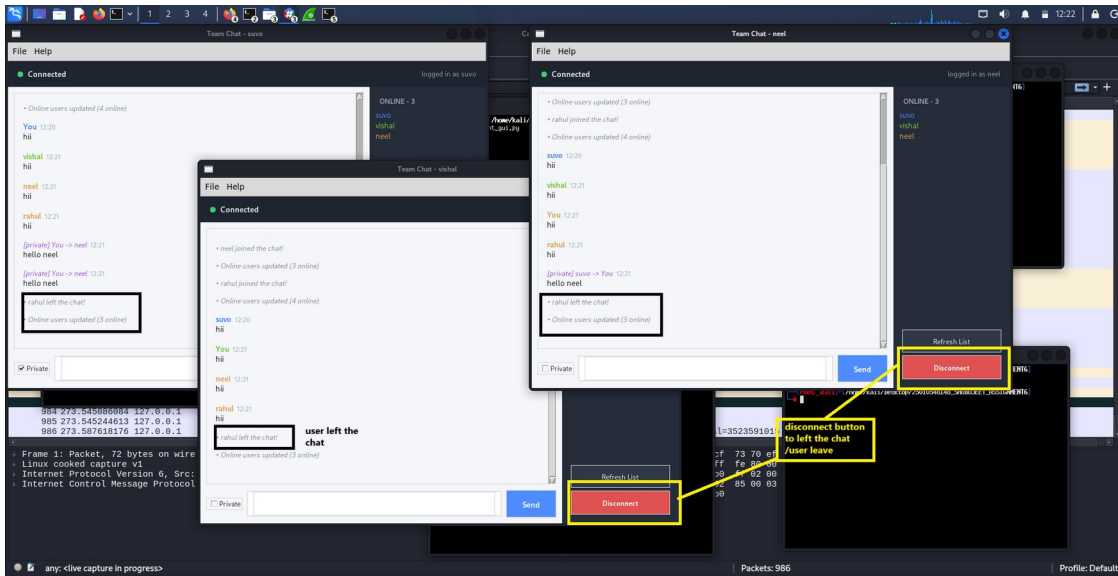


Figure 6.5: Client-initiated disconnect via the Disconnect button.

chat_history.csv recorded during this session:

timestamp,sender,receiver,message_type,message 12:20:50,suvo,ALL,broadcast,hii
12:21:07,vishal,ALL,broadcast,hii 12:21:13,neel,ALL,broadcast,hii
12:21:18,rahal,ALL,broadcast,hii 12:21:51,suvo,neel,private,hello neel

7. Wireshark Verification

Traffic was captured on interface 'any' with the display filter tcp.port == 5000 to isolate chat traffic from Mininet/OVS background traffic. The four scenarios below were captured from the same live test session as Section 6.

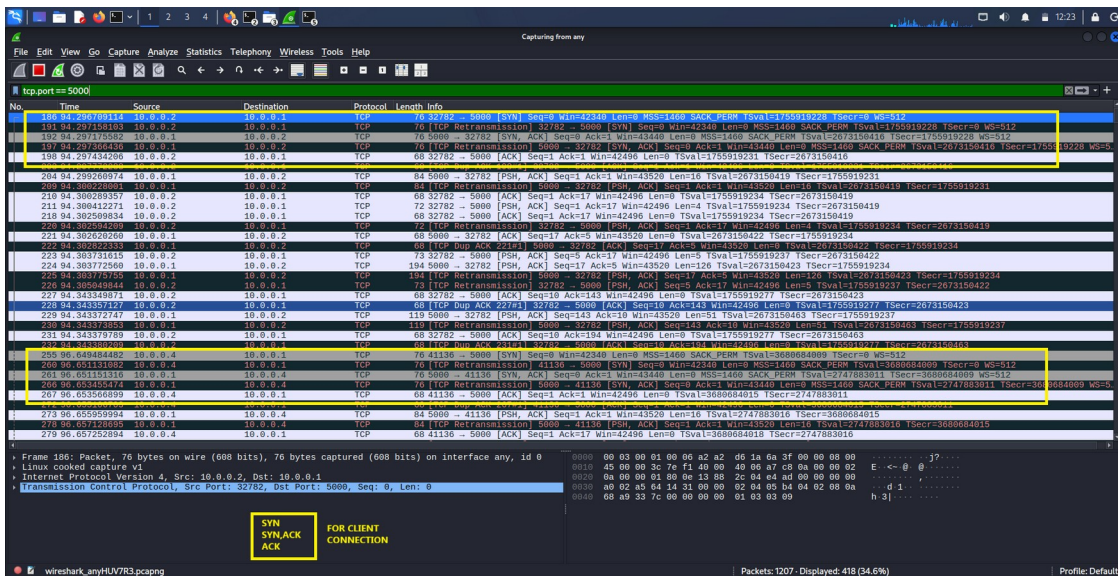


Figure 7.1: TCP three-way handshake (SYN, SYN-ACK, ACK) for a client connecting to the server.

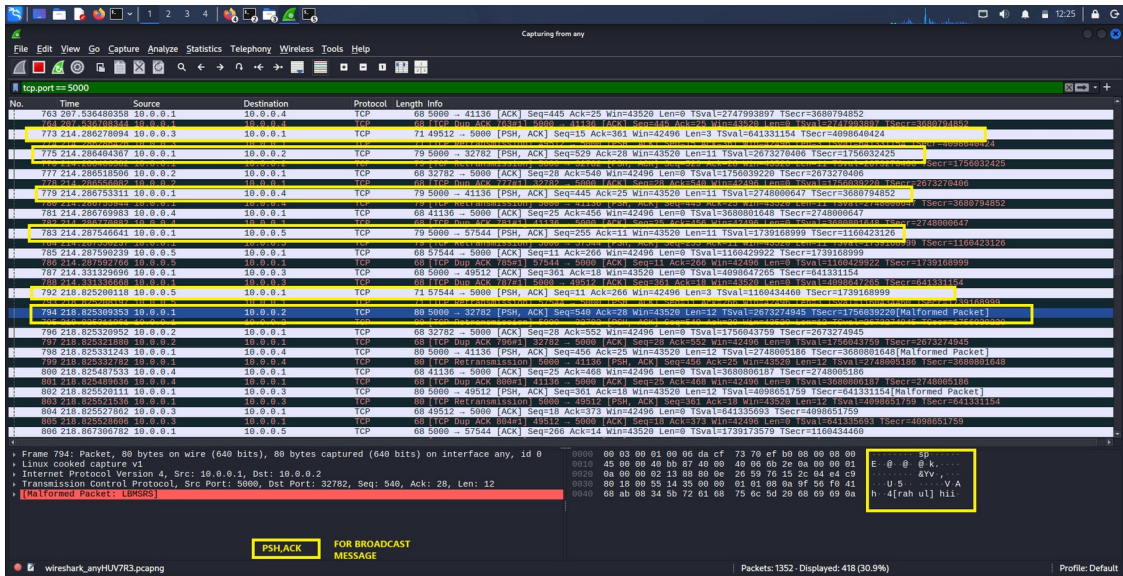


Figure 7.2: PSH,ACK packet carrying a broadcast chat message — hex payload decodes to "[rahu] hii", matching the chat_history.csv entry at 12:21:18.

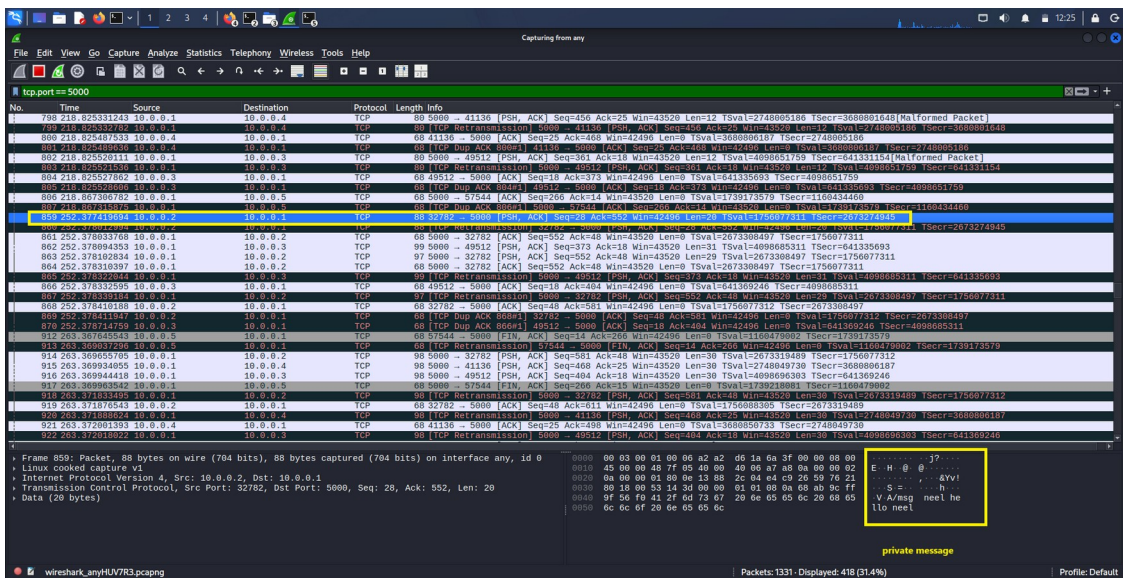


Figure 7.3: PSH,ACK packet carrying the private message command — hex payload decodes to "/msg neel hello neel", matching the private message from suvo to neel.

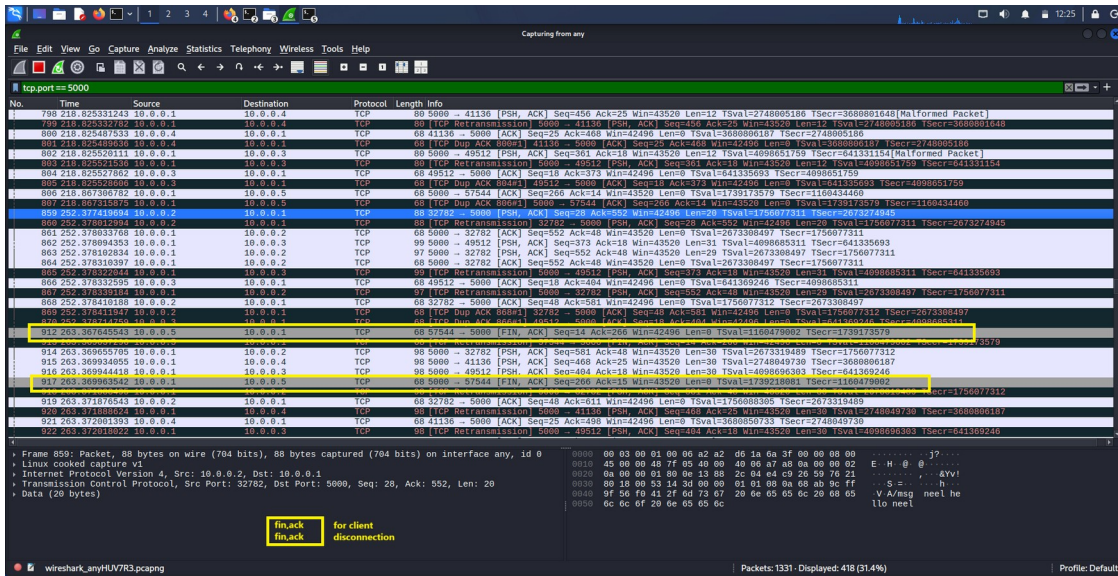


Figure 7.4: FIN,ACK exchange terminating the TCP connection on client disconnect.

Note: Wireshark's dissector occasionally labels chat payload segments as "[Malformed Packet]" in the packet list. This is expected — Wireshark attempts to interpret the raw TCP payload using a heuristic protocol guess, and the plain-text chat protocol used here does not match any registered dissector. It does not indicate a transmission error; the payload bytes themselves (visible in the hex pane) are intact and correctly delivered.

8. Reflection Answers

8.1 Why should networking logic and GUI code be separated?

Separating the two makes each side independently testable and reusable — the networking layer (ChatClientNetwork) has no knowledge of Tkinter and could be reused with a different frontend, while the GUI layer can be redesigned without touching socket logic. It also keeps responsibilities clear: the network layer only moves bytes and exposes them via a queue, while the GUI layer is solely responsible for rendering and user interaction, which makes the threading boundary between them easier to reason about and debug.

8.2 Why is a background thread required?

sock.recv() blocks until data arrives from the server. Tkinter runs a single-threaded event loop (mainloop()) that must keep processing UI events (keystrokes, button clicks, redraws) continuously; if recv() were called on that same thread, the entire window would freeze between messages. Running the receive loop on a separate daemon thread lets the GUI remain responsive at all times, with incoming data handed off safely through a thread-safe queue that the GUI polls periodically.

8.3 Which components from Assignment 5 were reused?

The entire server (server.py) was reused unmodified, including client management, broadcast and private messaging, the online user list, server statistics, and persistent chat history with reconnect replay. On the client side, only the exact same socket connect/send/rcv protocol was reused — the terminal input/output loop was replaced entirely by the Tkinter interface.

8.4 How did the GUI improve usability?

The terminal client required users to remember exact command syntax and offered no persistent view of who was online. The GUI surfaces the online user list continuously, colour-codes each sender for readability, separates private messages visually from broadcasts, and lets users send a private message by simply selecting a name from a list instead of typing a username correctly by hand — reducing input errors and making multi-user conversations far easier to follow.

8.5 Suggest one future enhancement.

Add TLS encryption to the socket connection so chat traffic is not sent as plaintext over the network, which the Wireshark captures in Section 7 currently show in full.

9. Conclusion

This assignment successfully replaced the terminal-based Assignment 5 chat client with a graphical Tkinter interface while reusing the server and socket protocol without modification. All required GUI components (login window, chat display, online user list, private/broadcast messaging, background message handling) were implemented and verified through live testing across four Mininet clients, with chat_history.csv and Wireshark packet captures confirming correct message delivery, private-message routing, and connection lifecycle events end-to-end.